OUTPUT ANALYSIS

Hospital Patient Management System • Doubly Linked List in C++

1. Time Complexity Analysis

Let **n** be the number of patient records currently stored in the doubly linked list.

Operation	Time Complexity	Reason
Add Patient	**O(n)**	Traverses from *head* to the last node before inserting the new patient.
Display Patients	**O(n)**	Visits and prints each patient node once.
Search Patient	**O(n)**	Linear search may examine every node if the patient is last or absent.
Discharge Patient	**O(n)**	Finding the patient takes O(n); pointer updates and deletion take O(1) after the node is found.

2. Space Complexity Analysis

The program stores patient data in dynamically allocated nodes. Each node contains patient fields and two pointers: *prev* and *next*.

Total Space Complexity	**O(n)**	The list contains n patient nodes, with a fixed amount of data stored in each node.
Auxiliary Space per Operation	**O(1)**	Display, search, and discharge use a fixed number of temporary pointers and variables. Adding a patient allocates one additional node.

Interpretation: The current implementation is linear-time for its main operations because it stores only a head pointer and must traverse the list to locate a target or the final node.